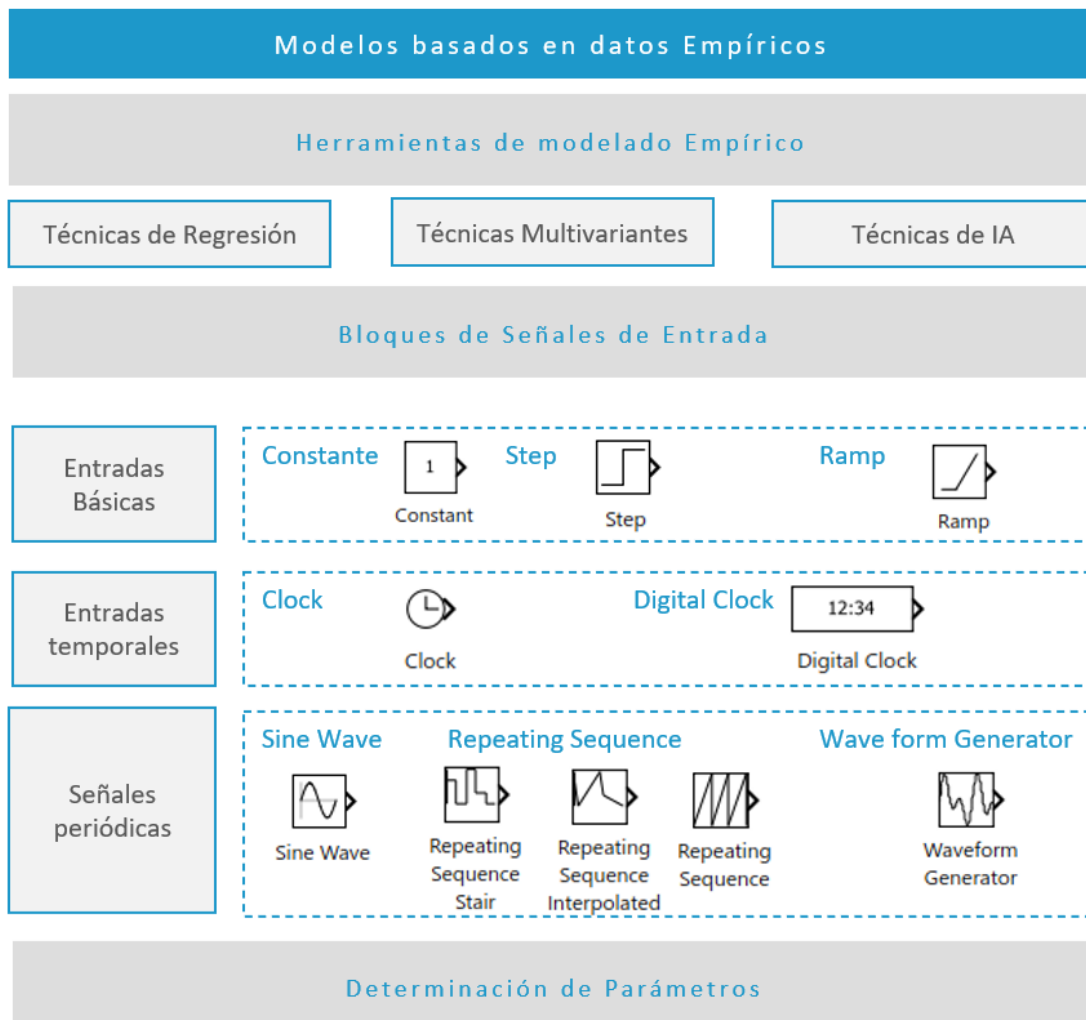


Modelado de Sistemas Dinámicos

Modelos basados en datos empíricos

Índice

Esquema.	2
Ideas clave	3
5.1 Introducción y objetivos	3
5.2 Herramientas para el modelado empírico	5
5.3 Bloques de introducción de datos en Simulink.	7
5.4 Determinación de parámetros	16
5.5 Referencias bibliográficas	21



5.1 Introducción y objetivos

El objetivo de todo modelo que desarrollemos es aproximarlos a la realidad. A menudo nos encontraremos con sistemas de los que no conocemos una ley física que los represente o bien la ley física que los representa es demasiado compleja y no la podemos implementar. En estos casos recurriremos a datos empíricos para realizar el modelado. Conocemos este tipo de modelado como modelos de caja negra puesto que no conocemos por qué sucede únicamente el qué sucede. Se suelen tratar de modelos creados para una situación concreta y no siempre son generalizables.

Vamos a distinguir dos tipos de modelado empírico. El primer tipo de modelado es aquel que está basado en leyes físicas pero que requiere del ajuste de parámetros para funcionar correctamente, se conocen como modelos cuasi-empíricos. El segundo tipo es el basado estrictamente en los datos y no conocemos ningún tipo de información sobre el sistema.

En este tema profundizaremos en los métodos empíricos y veremos como introducir nuevos datos en Simulink. También introduciremos la herramienta de optimización de parámetros que incluye Simulink.

Un ejemplo: El modelado Geopotencial

El modelo geopotencial es un modelo utilizado para describir la fuerza de la gravedad ejercida por la Tierra considerando que ésta no es una esfera perfecta sino un geoide.

Si partimos de la ley de la gravitación universal de Newton, sabemos que:

$$\vec{F} = -G \frac{m_1 m_2}{r^2} \vec{r}. \quad (1)$$

Dónde F es la fuerza que recibe un cuerpo, G es la constante de gravitación universal ($6.674 \cdot 10^{-11} \text{ m}^3 \text{kg}^{-1} \text{s}^{-2}$), m_1 y m_2 son las masas de los dos cuerpos que se están considerando, $\vec{r}$ es el vector que une los dos cuerpos y $r = \|\vec{r}\|$.

Esta fórmula es cierta si consideramos un objeto de masa puntual, esto es, infinitesimalmente pequeño. Sin embargo, si lo aplicamos a un objeto que tenga un determinado volumen, la fórmula deja de ser cierta. Sin embargo, descomponiendo en porciones infinitesimales podemos llegar a la fórmula:

$$\vec{F} = -Gm_2 \int_V \frac{\rho}{r^2} \vec{r} \, dxdydz.$$

Donde $\rho = \rho(x, y, z)$ es la densidad, con lo que la masa del planeta será $m_1 = \int_V \rho \, dxdydz$.

Si consideramos que el planeta es una esfera de densidad constante, entonces podemos considerar que la fuerza que ejerce el planeta sobre una masa viene dada por la Ecuación 1. Intuitivamente, podemos pensar que las fuerzas que nos desviarían fuera del vector entre las dos masas se compensan. Algo parecido ocurre con las fuerzas que están más alejadas o más cercanas al centro de masas del planeta. Se puede encontrar una demostración con técnicas geométricas elementales de dicho resultado en (Borghi, 2014).

Con lo que se puede describir el potencial terrestre por:

$$u = \frac{-GM}{r}$$

de forma que conocemos la relación: $\vec{F} = \left(\frac{\partial^2 u}{\partial x^2}, \frac{\partial^2 u}{\partial y^2}, \frac{\partial^2 u}{\partial z^2} \right)$.

Ahora bien, si queremos tener en cuenta un objeto que no sea perfectamente esférico, dicha aproximación no será válida. Se pueden intentar distintas alternativas. Por ejemplo considerar la tierra un elipsoide o un elipsoide a capas. Sin embargo, tampoco tendremos en cuenta la orografía terrestre ni los posibles cambios en su densidad. Está claro, que tampoco es viable calcular la integral del volumen. Así pues, debemos acudir a otra solución. Esta solución pasa por el camino contrario. Vamos a utilizar el movimiento real de una nave en el espacio para conocer el campo gravitatorio terrestre. La idea será ampliar la ecuación 1 añadiendo fuerzas que dependan de la posición en la que se encuentre la masa atraída por el planeta.

Para ello se emplean los esféricos armónicos, estos son funciones de la forma:

$$g(r, \theta, \varphi) = \frac{1}{r^{n+1}} P_n^m(\sin \theta) \cos m\varphi, \quad 0 \leq m \leq n, \quad n = 0, 1, 2, \dots$$

$$h(r, \theta, \varphi) = \frac{1}{r^{n+1}} P_n^m(\sin \theta) \sin m\varphi, \quad 1 \leq m \leq n, \quad n = 1, 2, \dots$$

con (r, θ, φ) las coordenadas esféricas del punto (x, y, z) y $P_n^m(x)$ las funciones de Legendre asociadas.

Este tipo de funciones son de especial interés puesto que resuelven una ecuación en derivadas parciales de la forma:

$$\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} + \frac{\partial^2 u}{\partial z^2} = 0$$

Así pues, podemos describir el potencial terrestre como

$$u = -\frac{\mu}{r} \left(1 + \sum_{n=2}^{N_z} \frac{J_n P_n^0(\sin \theta)}{\left(\frac{r}{R}\right)^n} + \sum_{n=2}^{N_t} \sum_{m=1}^n \frac{P_n^m(\sin \theta) (C_n^m \cos m\varphi + S_n^m \sin m\varphi)}{\left(\frac{r}{R}\right)^n} \right).$$

Donde $\mu = G \cdot m_1$, R es el radio de la tierra y los valores J_n , C_n^m y S_n^m son coeficientes que dependerán del planeta. Observemos que si todos los coeficientes son nulos, estamos en el caso de un planeta perfectamente esférico.

El siguiente paso para conocer la fuerza de atracción, consiste en encontrar de forma empírica la posición de una nave espacial orbitando el planeta. Después podemos ajustar sucesivamente los coeficientes de forma que el movimiento simulado coincida con el movimiento observado.

En este tema vamos a ver diferentes técnicas que nos permiten realizar este tipo de aproximaciones.

5.2 Herramientas para el modelado empírico

El modelado empírico consiste en la generación de modelos que se basan en datos obtenidos empíricamente y no en leyes físicas. El origen de estas herramientas son las técnicas de regresión. Posteriormente, las técnicas de regresión fueron evolucionando hacia técnicas de agrupación así como un conjunto de herramientas que hoy en día se conocen como técnicas multivariantes. La evolución del campo en los últimos años ha llevado a la incorporación de herramientas de inteligencia artificial para realizar estudios de modelado empírico.

Técnicas de regresión

Una de las técnicas que más se emplean cuando se observan datos empíricos es la regresión lineal. El problema consiste en encontrar los coeficientes a y b dentro del binomio

$$f(t) = at + b,$$

de forma que los datos que se han tomado durante el experimento y la aproximación dada por la función sea la mínima posible.

También nos podemos encontrar con regresiones que dependan de más de un parámetro. Entonces nos encontraremos ante regresión multilíneal. En este caso buscaremos funciones de la forma

$$f(x_1, \dots, x_n) = a_0 + a_1x_1 + \dots + a_nx_n$$

Dentro de la regresión también se incluyen técnicas de regresión no lineal, como puede ser la regresión a funciones exponenciales, logarítmicas u otras funciones especiales.

Para realizar este cálculo se suele emplear mínimos cuadrados.

Técnicas Multivariantes

Las técnicas multivariantes parten de la idea de la regresión lineal en el sentido que también buscan aproximar ciertas funciones a partir de los datos medidos en la realidad. Estas técnicas representan una evolución de la regresión lineal. Dentro de estas técnicas se engloban:

- Análisis de varianza (comúnmente conocido como ANOVA). Este análisis consiste en considerar que las muestras se pueden escribir de la forma:

$$y_{ijk\dots} = \mu + f(i, j, k, \dots) + \varepsilon.$$

Donde $y_{ijk\dots}$ es la medida realizada, μ es una constante del modelo, f una relación entre las variables independientes y ε el error tomado en la medida. A partir de estos datos y siempre que se verifiquen algunas hipótesis previas, podremos encontrar valores para resolver el modelo.

- ▶ Transformada de Fourier. La transformada de Fourier se utiliza especialmente cuando las mediciones que se han tomado son periódicas o cercanas a serlo. En este caso se identifican las amplitudes y frecuencias características de los datos. Los modelos que se obtienen son en forma de funciones trigonométricas, senos y cosenos.
- ▶ Análisis factorial o de factores principales. Se trata de una técnica para ver cuáles son los principales elementos que intervienen en un modelo y que se deberán priorizar en el modelado. Normalmente se emplean en conjunción con otras técnicas una vez se han descartado los componentes principales.

Técnicas de Inteligencia Artificial

En los últimos años, se ha incrementado el número de modelos que trabajan con técnicas de inteligencia artificial. La principal técnica que se implementa es la de las redes neuronales. Las redes neuronales intentan replicar la forma de pensar del ser humano. Se realizan un conjunto de conexiones entre neuronas. Cada neurona activa la siguiente etapa y así sucesivamente hasta llegar a la etapa final. En dicha etapa se produce la salida del modelo.

5.3 Bloques de introducción de datos en Simulink

Cómo hemos visto, en el proceso de modelado es importante realizar una buena introducción de los datos en nuestro modelo. Para ello, Simulink incorpora multitud de bloques que nos pueden resultar de interés. En esta sección vamos a estudiar las principales formas de introducir valores de entrada al modelo.

A lo largo del curso hemos introducido algunos elementos. Vamos a detallar algunas entradas y a ampliar los detalles de las mismas. Los bloques de los que vamos a hablar a continuación se encuentran en la pestaña de *Sources* del *Library Browser*.

Bloques de señales de entrada básicas

Los bloques de entrada básicos son aquellos que nos permiten introducir una señal sencilla. Distinguiremos tres tipos, los bloques de entrada constante, los bloques de escalón y los bloques de rampa.

- ▶ **Bloque Constant.** Se trata del bloque básico por excelencia. Emite una señal constante a lo largo de toda la simulación. Podemos fijar el valor de inicio o, alternativamente, tomar una variable del espacio de trabajo de Matlab. Al ejecutar la simulación la variable cogerá dicho valor. La curva azul en la figura 1 se ha generado con un bloque constante fijado a 1.5.
- ▶ **Bloque Step:** Este bloque mantiene un valor constante hasta un instante de tiempo, momento en el que se actualiza el a un nuevo valor constante. Una vez se ha actualizado el valor no se puede volver a actualizar. En la configuración del bloque encontraremos tres parámetros fundamentales:
 - *Step Time:* instante t_0 de tiempo en el que se produce el cambio de constante.
 - *Initial Value:* Valor constante c_0 al inicio de la simulación y hasta llegar al valor definido por *Step Time*.
 - *Final value:* Valor constante c_f después del instante de tiempo definido por *Step Time*.

La señal queda definida por:

$$\begin{cases} c_0 & \text{si } t < t_0, \\ c_f & \text{si } t \geq t_0. \end{cases}$$

Usaremos este tipo de bloque para simular el encendido o apagado de una cierta maquinaria o efecto dentro de nuestra simulación. La señal naranja de la Figura 1 se ha generado con un bloque Step con *Step Time* fijado a 1, *Initial value* fijado a 0 y *Final value* fijado a 1.

- ▶ **Boque Ramp:** Este bloque tiene un efecto parecido al anterior. Se inicia con un valor constante, llegado un instante de tiempo se activa el bloque y se incrementa de forma lineal hasta que finaliza la simulación. Como en el bloque anterior tenemos tres parámetros fundamentales:
 - *Slope:* nos indica la pendiente de la señal s .
 - *Start time:* nos indica el instante t_0 en el que la señal empieza a crecer.

- *Initial output*: nos indica el valor constante que tiene la señal al inicio de la simulación c_0 .

La señal queda definida por:

$$\begin{cases} c_0 & \text{si } t < t_0, \\ c_0 + s(t - t_0) & \text{si } t \geq t_0. \end{cases}$$

Este tipo de bloques se suelen emplear para indicar una cantidad que empieza a crecer pasado un cierto tiempo, por ejemplo la demanda de un cierto producto de mercado. La señal amarilla de la Figura 1 se ha generado usando un bloque Ramp configurado con *Slope* de 0.5, *Start Time* de 2 y *Initial Output* de 0.2.

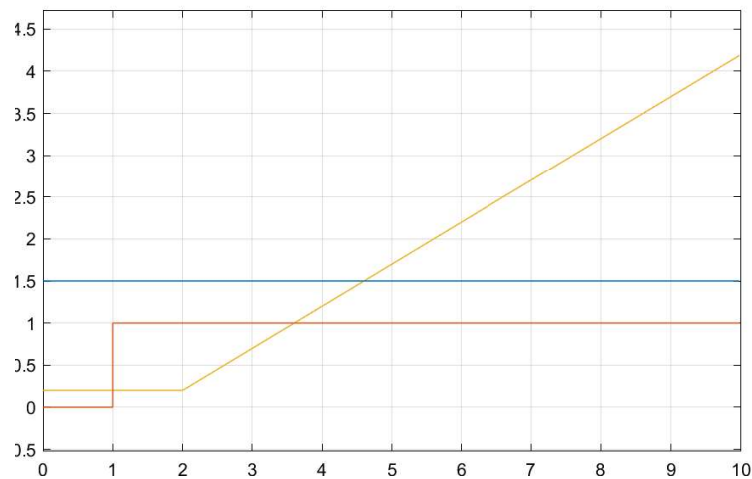


Figura 1: Señales generadas por los bloques Constant (señal azul), Step señal naranja y Ramp señal amarilla.

Bloques de medición del tiempo

Los siguientes bloques nos van a permitir conocer el tiempo en el que se encuentra la simulación.

- Bloque Clock: este bloque nos devuelve el tiempo de simulación de forma continua. A medida que va avanzando la simulación también avanza la señal emitida por el bloque.
- Bloque Digital Clock: al igual que el bloque anterior también devuelve el tiempo de simulación en la señal. Sin embargo, la actualización se realiza de forma discreta. Esto se puede emplear para simular ciertas situaciones en las que queremos

que la lectura del tiempo sea constante durante un tiempo. Podemos controlar el tiempo de actualización realizando una doble pulsación sobre el bloque y modificar el valor *Sample time*. El valor introducido será la frecuencia con la que se actualiza la señal de tiempo.

En la Figura 2 podemos ver las señales emitidas por los dos bloques. La señal azul proviene de un bloque de Digital Clock configurado con refresco cada unidad de tiempo. La señal naranja se ha generado con un bloque Digital Clock con refresco cada media unidad de tiempo. Finalmente, la señal amarilla está generada por un bloque Clock.

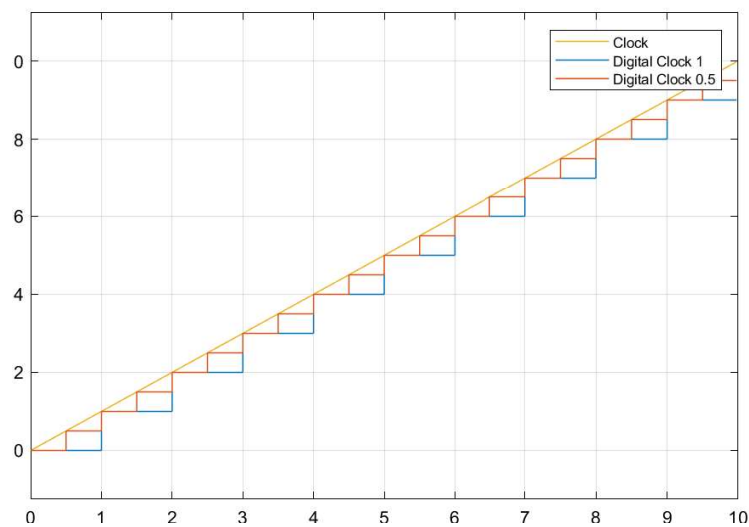


Figura 2: Señales generadas por los bloques Clock(señal amarilla) y Digital Clock señales azul (con refresco cada unidad de tiempo) y naranja (con refresco cada media unidad de tiempo).

Señales de contador

Las señales de contador nos permiten realizar recuentos. En todos los casos podemos configurar la cantidad de tiempo para actualizar el contador. Por defecto el *Sample Time* está configurado a -1 . Esto indica que se actualiza el contador en función de la actualización del resto del sistema..

- El bloque Counter: realiza un recuento, cada *Sample Time* unidades de tiempo se incrementa la señal en uno.

- El bloque **Lim Counter**: realiza un recuento limitado a una cierta cantidad. Cuando se alcanza dicha cantidad el recuento se resetea a 0 y vuelve a empezar.

El interés de este tipo de bloques reside en control sobre el número de pasos que ha realizado el integrador. El contador limitado también nos puede servir para resetear alguna variable después de un número determinado de eventos. Ya sea pasos de integración o un número de intervalos de tiempo concreto.

En la figura 3 se muestran las señales emitidas por los bloques **Counter** y **Lim Counter**. El primero se ha configurado para realizar un incremento cada 0.6 unidades de tiempo. El segundo bloque se ha dejado con el parámetro de actualización por defecto en -1 , de forma que se actualiza en función de la actualización definida por el **Counter**.

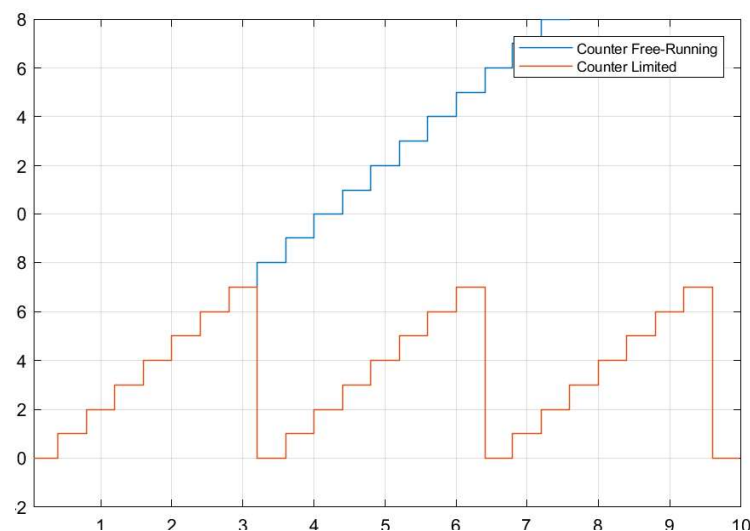


Figura 3: Señales generadas por los bloques **Counter**(señal azul) y **Lim Counter** señal naranja.

Observación: Si se programa un bloque de contador con tiempos de actualización idénticos a los de un bloque **Digital Clock** con el mismo tiempo de actualización.

Señales periódicas

Estos bloques nos permiten incorporar una señal de forma que se vaya repitiendo a lo largo de la simulación. Existen varios bloques que nos permiten poner señales en modo de repetición. Ya hemos visto el contador limitado que va repitiendo la señal

cada cierto número de valores contados. Los siguientes bloques se pueden configurar para reproducir el resultado deseado.

► Bloque Sine Wave: nos permite definir una señal sinusoidal de la forma:

$$s(t) = A * \sin(\omega * t + \varphi) + b.$$

Los valores de A , ω , φ y b son constantes que podremos configurar realizando una doble pulsación en el bloque. Se corresponden a las configuraciones:

- *Amplitude*: la amplitud A de la señal sinusoidal.
- *Bias*: la constante b que añadimos a la señal.
- *Frequency*: la frecuencia de la señal ω .
- *Phase*: la fase de la señal φ .

Con estos valores se puede generar cualquier señal que sea trigonométrica, incluido una señal cos. Para ello, deberemos tener en cuenta que $\cos(t) = \sin(t + \pi/2)$. La Figura 4 muestra en color naranja la señal generada por una señal con $A = \omega = 1$, $b = 0$ and $\varphi = \pi/2$.

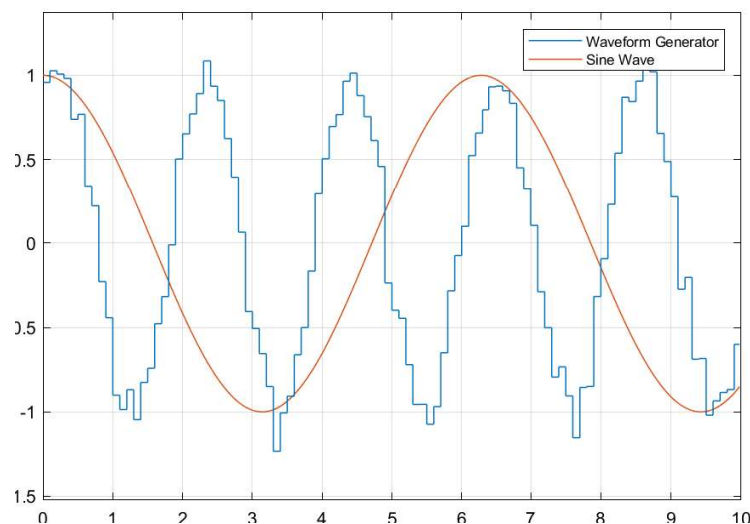


Figura 4: Señales generadas por los bloques Sine Wave(señal naranja) y Wave Form Generator señal azul.

► Bloque Repeating Sequence: Este bloque nos permite introducir una secuencia de valores s_i y un conjunto de tiempos t_i , ambos conjuntos deben ser de la misma longitud. El bloque emite una señal de intensidad s_i en el tiempo t_i . en los puntos

intermedios realiza una interpolación lineal. Cuando se agotan los puntos, vuelve a empezar la secuencia. El bloque admite los siguientes parámetros de configuración:

- *Time Values*: vector de tiempos de los puntos de interpolación t_i .
- *Output Values*: vector de valores s_i por los que pasará la señal emitida.

En la Figura 5 se muestra la salida del bloque *Repeating Sequence* en color amarillo.

- **Bloque *Repeating Sequence Interpolated***: Este bloque permite realizar una acción parecida a la del bloque anterior. También se introducen un conjunto de valores s_i y tiempos t_i , la señal emitida pasará en tiempo t_i por la intensidad s_i . También podremos seleccionar el tipo interpolación que nos interesa. Los parámetros de configuración son los siguientes:

- *Vector of output values*: valores s_i que emitirá el bloque.
- *Vector of time values*: valores t_i en los que se emitirá la señal con intensidad s_i .
- *Lookup Method*: permite seleccionar el método de interpolación que se utilizará. Podremos seleccionar los métodos siguientes:
 - *Interpolation - Use end values*: realiza una interpolación lineal entre el punto t_i y t_{i+1} .
 - *Use Input Nearest*: realiza la salida del bloque con una función escalón. El valor de la salida s_i que se toma es el asociado al t_i más cercano.
 - *Use Input Below*: realiza la salida del bloque con una función escalón. El valor de la salida s_i se corresponde con el del t_i mayor más pequeño a t .
 - *Use Input Above*: realiza la salida del bloque con una función escalón. El valor de la salida s_i se corresponde con el del t_i menor más grande a t .

Podemos configurar el bloque de forma que reproduzca el bloque *Repeating Sequence* reproduciendo el vector de valores temporales y de salida con el método de interpolación *Interpolation - Use end values*. En la Figura 5 podemos ver la señal emitida por el bloque con un vector de valores $[3 \ 1 \ 4 \ 2 \ 1]$ y valores de tiempo $[0 \ 0.5 \ 1 \ 1.2 \ 2]$ con el método de interpolación *Interpolation* en color azul.

- **Bloque *Repeating Sequence Stair***: Al igual que los dos bloques anteriores este bloque permite emitir una señal de forma periódica. La señal siempre será escalonada e irá alternando entre los valores configurados siempre con el mismo intervalo temporal. Se configura con dos parámetros:

- *Vector of output values*: es un vector que contiene las señales de salida s_i .
- *Sample Time*: el tiempo durante el que se tomará la señal constante.

Cabe destacar que con este bloque no se puede configurar la longitud de tiempo emitido. Será igual para todos los valores. Podemos ver una señal configurada con cambios de tiempo de 0.5 y vecotr de valores de salida [3 1 4 1] . ' en la Figura 5.

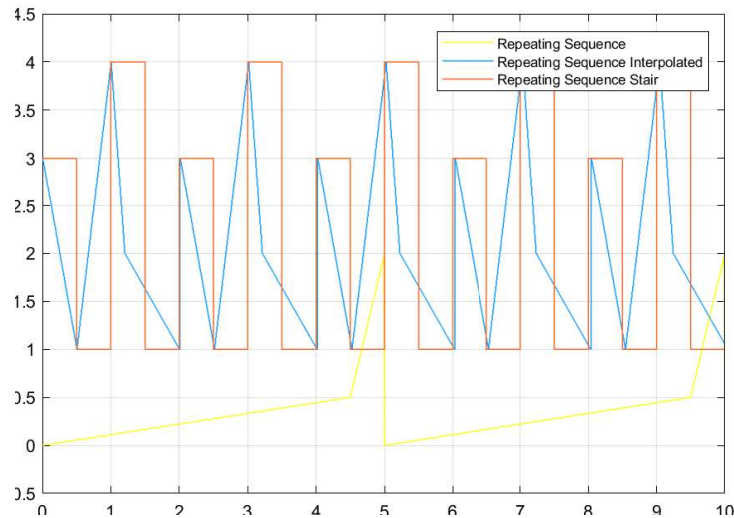


Figura 5: Señales generadas por los bloques Repeating Sequence(señal amarilla), Repeating Sequence Interpolated (señal azul) y Repeating Sequence Stair señal naranja.

► **Wave Form Generator:** El último bloque que vamos a introducir es el bloque de generación de ondas. Este bloque nos permitirá introducir ondas con distintas características. Su funcionamiento es distinto a los demás. Al abrir la ventana de configuración podremos añadir distintas señales de salida y deberemos seleccionar una de ellas. Esto nos permitirá configurar distintos escenarios y situaciones. A continuación, deberemos definir cómo será la salida de la señal. Dispondremos de un espacio de texto para la configuración. En dicho espacio deberemos introducir las señales como si de funciones se trataran. Podemos añadir señales de tipo:

- **Constante:** Para ello deberemos introducir cualquier valor real o variable definida en el Workspace.
- **Ruido Gausiano:** se introduce a través del comando:

`gaussian(media,varianza,semilla).`

En cada lectura de la señal genera un valor aleatorio siguiendo una distribución normal de media `media`, varianza `varianza` y partiendo de la semilla inicial `semilla`.

- **Pulso:** la introduciremos con el comando:

`pulse(amplitud, inicio, duración).`

Generará un pulso de amplitud `amplitud` que se iniciará en el tiempo `inicio` y durará `duración` unidades de tiempo.

- Diente de Sierra: genera una señal de diente de sierra a través del comando:

`sawtooth(amplitud, frecuencia, fase).`

Se generará una diente de sierra desde `-amplitud` hasta `amplitud` con picos cada `frecuencia` e iniciando en `fase`.

- Sinusoidal: genera una señal sinusoidal a través del comando:

`sin(amplitud, frecuencia, fase).`

Se generará una función seno con amplitud `amplitud` frecuencia `frecuencia` e iniciando en `fase`.

- Cuadrada: se genera una señal cuadrada mediante la instrucción:

`square(amplitud, frecuencia, porcentaje).`

La señal oscilará entre el máximo y el mínimo definido en `amplitud` con la frecuencia marcada por `frecuencia`. El valor `porcentaje` definirá el porcentaje de tiempo en el que la señal está activa (con valor `+amplitud`), deberá ser un valor entre 0 y 100.

- Salto: podemos introducir una función de salto mediante la llamada a:

`step(tiempoSalto, valorInicial, valorFinal).`

La señal tomará la intensidad definida en `valorInicial` hasta que se llegue al tiempo definido por `tiempoSalto` momento en el que tomará el valor `valorFinal`. Funciona de forma equivalente a un bloque `step`.

Este tipo de bloque permite generar señales parecidas a las del resto de elementos. Tiene dos ventajas principales. Por un lado, nos permite realizar la suma de dos o más señales en una misma entrada. A modo de ejemplo, en la Figura 4 se muestra la salida de este bloque combinando una sinusoidal y una señal aleatoria en color azul. De esta forma, podemos introducir ruido a la señal determinada. La segunda ventaja consiste en generar señales que nos pueden interesar según el tipo de modelado que hemos realizado. Así pues, podemos plantear tres posibles modelos, dejarlos introducidos e ir escogiendo el modelo en función de las pruebas y tests que queremos realizar. Por contra, el modelo no resulta tan claro y se debe abrir el bloque para saber qué tipo de señal se ha implementado.

Por defecto, la salida de la señal es una función escalonada. Es posible definir la longitud de los escalones modificando el valor *Sample Time* de la pestaña *Signal Attributes* de la ventana de configuración del bloque.

El vídeo “El bloque Wave Form Generator” profundiza sobre este bloque y muestra algunos de los resultados que se pueden implementar con él.



Accede al vídeo: El bloque Wave Form Generator

5.4 Determinación de parámetros

Una vez vistas las diferentes formas de introducir señales en Simulink, vamos a trabajar sobre cómo determinar los parámetros de un sistema. Ya hemos comentado que es compleja y resulta delicada. Por suerte, podemos incorporar una herramienta a Simulink para que nos ayude en el proceso: el estimador de parámetros.

Lo primero que deberemos hacer es instalar el Add-on adecuado. Para ello, abriremos el *Add-On Explorer* igual que mostramos en el vídeo de instalación de Simulink y buscaremos e instalaremos el Add-on *Simulink Design Optimization*. Esto nos añadirá un conjunto de bloques a Simulink bajo la pestaña *Simulink Design Optimization*, pero lo que más nos interesa es que nos añadirá una nueva app para utilizar en nuestros modelos, el *Parameter estimator*. Para acceder a ella deberemos tener abierto un modelo de Simulink. En la parte superior abriremos la pestaña *Apps* y encontraremos distintas aplicaciones o complementos para usar junto con nuestro modelo. Por ahora vamos a abrir el *Parameter estimator*.

Esta funcionalidad nos permite determinar, a partir de unos valores de test, los parámetros que satisfacen los valores de test.

El proceso sigue los siguientes pasos:

1. Pulsamos sobre *New Experiment*. Esto nos abrirá una ventana de configuración para diseñar un nuevo experimento de estimación:
 - a) Seleccionamos las salidas de nuestro experimento. Si el modelo tiene salidas definidas, estas se seleccionarán de forma automática. Si no las tiene, entonces

deberemos seleccionarlas pulsando en *Select Measured Output Signals* y pulsar las señales que queremos usar como salida.

- b) Una vez seleccionadas las señales debemos indicar qué valor esperamos de dicha señal. Para ello, debemos introducir una matriz con los tiempos y las magnitudes esperadas para la señal en el cuadro de texto justo por debajo de la definición de la señal.
- c) Si el modelo tiene entradas, deberemos introducir la señal de entrada al modelo.
- d) De forma opcional, también podemos definir las condiciones iniciales del problema para resolver dicho experimento.
- e) El último paso obligatorio consiste en definir los parámetros que deseamos estimar. Para ello usaremos la última sección de la ventana que se nos ha abierto.
- f) Completaremos la configuración pulsando en el botón *Plot and Simulate*. Esto ejecutará una simulación del modelo con el valor inicial del parámetro. Y nos enseñará el valor simulado comparado con el valor esperado.

2. Una vez definido el experimento debemos seleccionar el tipo de estimación que queremos realizar. Usualmente un buen estimador es el de la suma de los errores al cuadrado:

$$err = \sum_{i=1}^n (s_i - r_i)^2.$$

donde s_i son los valores encontrados en la simulación y r_i son los valores reales introducidos al sistema.

3. Pulsa sobre el botón *Estimate*. Matlab empezará a ejecutar el modelo para ir realizando estimaciones sucesivas del mismo. Se nos abrirá una ventana de reporte en la que aparecerán el número de iterados realizados, el número de evaluaciones funcionales y el valor de *err* en cada iterado. Dependiendo de la complejidad del problema esto puede costar bastante tiempo.
4. Una vez finalizada la estimación Matlab actualizará las variables que se han optimizado. También podremos visualizar gráficamente la evolución de los parámetros y el valor del error en cada iterado. Para eso, podemos añadir nuevos gráficos pulsando sobre *Add Plot* y seleccionando el gráfico que deseamos. Esto será interesante para estudiar la velocidad de convergencia o si el algoritmo ha quedado atascado en algún mínimo local.

Ejercicio 1. Considera el modelo de estudio de la bala que sala impulsada de un cañón a una velocidad de 30ms^{-1} con un ángulo de 30° y se mueve atraída por la gravedad y presenta una cierta fricción aerodinámica. Sabemos que la densidad

del aire es de 0.074, el área que presenta la bala es de 0.2, sin embargo desconocemos el coeficiente aerodinámico C_D . Hemos realizado mediciones de la altura y distancia recorrida por la bala a lo largo del tiempo y hemos obtenido los valores del fichero `xyCanon.mat`.

Determina el coeficiente aerodinámico C_D de la bala.

Para resolver este problema vamos a partir del modelo que empleamos en el tema anterior. Antes de abrir el modelo vamos a importar los datos del fichero `xyCanon.mat` a Matlab y vamos a crear una variable `coefD`. Como no sabemos su valor, la vamos a fijar a 0.

A continuación abriremos el fichero y lo vamos a modificar de forma que se identifique el coeficiente C_D en una única variable. Este paso no es necesario, pero nos ayudará a clarificar el proceso. Para realizarlo vamos a editar el valor de los dos bloques `Gain` que determinan la fricción aerodinámica eliminando el término correspondiente a C_D . A continuación añadiremos un bloque `Constant` para fijar el valor de C_D , lo editaremos e introduciremos el nombre de la variable que queremos optimizar, en nuestro caso `coefD`. Además, añadiremos a cada bucle de fricción un bloque producto realizar el producto por C_D . Al final el modelo nos quedará según se indica en la Figura 6.

El siguiente paso será abrir la aplicación *Parameter estimator* que se encuentra en la pestaña *Apps*. La imagen superior izquierda de la Figura 7 muestra la ventana de configuración del estimador de parámetros. En esta ventana configuramos nuestro experimento de estimación:

1. Pulsamos sobre *New Experiment*. Como nuestro modelo no tiene valores de entrada ni de salida no nos aparece ninguna señal por defecto.
 - a) Pulsamos sobre *Select Measured Output Signals* (ver imagen Superior Derecha de la Figura 7 y a continuación pulsamos sobre las señales x e y de nuestro modelo. Pulsamos Ok para confirmar las señales.
 - b) A continuación nos aparecerán dos cuadros de texto, uno debajo de cada señal con dos corchetes (`[]`) pre-introducidos. En estos corchetes debemos introducir la matriz de tiempos y posiciones correspondiente a las señales que hemos importado a través del fichero de datos. En la variable x pondremos `[t,x]` y en la variable y pondremos `[t,y]`. Una vez introducido nos aparecerá el número de señales y la dimensión que tiene la señal.

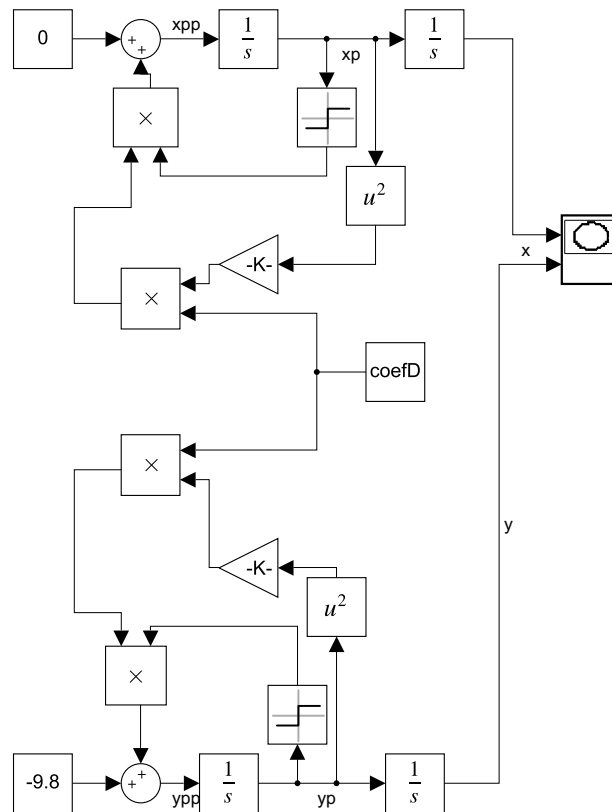


Figura 6: Modelo para la resolución del ejercicio 1 para determinar el coeficiente aerodinámico de una bala de cañón.

- c) La sección de condiciones iniciales la vamos a dejar vacía. Más adelante veremos ejemplos sobre cuando nos interesa modificar los estados.
- d) Finalmente pulsaremos sobre *Select Parameters* para indicar cuáles son los parámetros de nuestro sistema. Solo tenemos el parámetro *coefD*, así pues, lo seleccionamos y pulsamos ok. Desplegando la flecha negra podemos fijar valores máximos y mínimos para el parámetro así como el valor inicial por el que empezaremos a estimar. Vamos a fijar el mínimo del parámetro a 0 puesto que no puede ser negativo.

La configuración del experimento debería quedar como se muestra en la imagen superior derecha de la Figura 7. Pulsamos sobre *Plot & Simulate* y seguidamente sobre *Ok*. Nos aparece la simulación y el valor esperado en pantalla. Ahora, la ventana debería quedar como se muestra en la imagen inferior izquierda de la Figura 7. Podemos ver como inicialmente las curvas para x e y son parecidas, pero a medida que avanza el tiempo van divergiendo.

- Para realizar la estimación, pulsamos sobre el botón *Estimate*. Esto iniciará un conjunto de iteraciones por parte de Simulink. Al cabo de unos segundos (que pueden ser horas si el modelo es complejo), Simulink terminará con la estimación. Esta acción también generará una nueva gráfica con el valor estimado del parámetro en cada iteración. Podemos ver que inicialmente el parámetro C_D era 0, y en sucesivos iterados se ha ido acercando a 0.7. En la parte izquierda también podemos ver que la señal medida y la simulada ahora coinciden. La imagen inferior derecha de la Figura 7 nos muestra el resultado de la simulación, junto con los iterados y el valor de la función de coste.
- Podemos realizar el análisis de la evolución de la función de coste añadiendo una nueva gráfica. Para ello pulsamos sobre *Add Plot* y a continuación sobre *Estimation Cost*. Es necesario realizar esta acción antes de empezar la simulación.

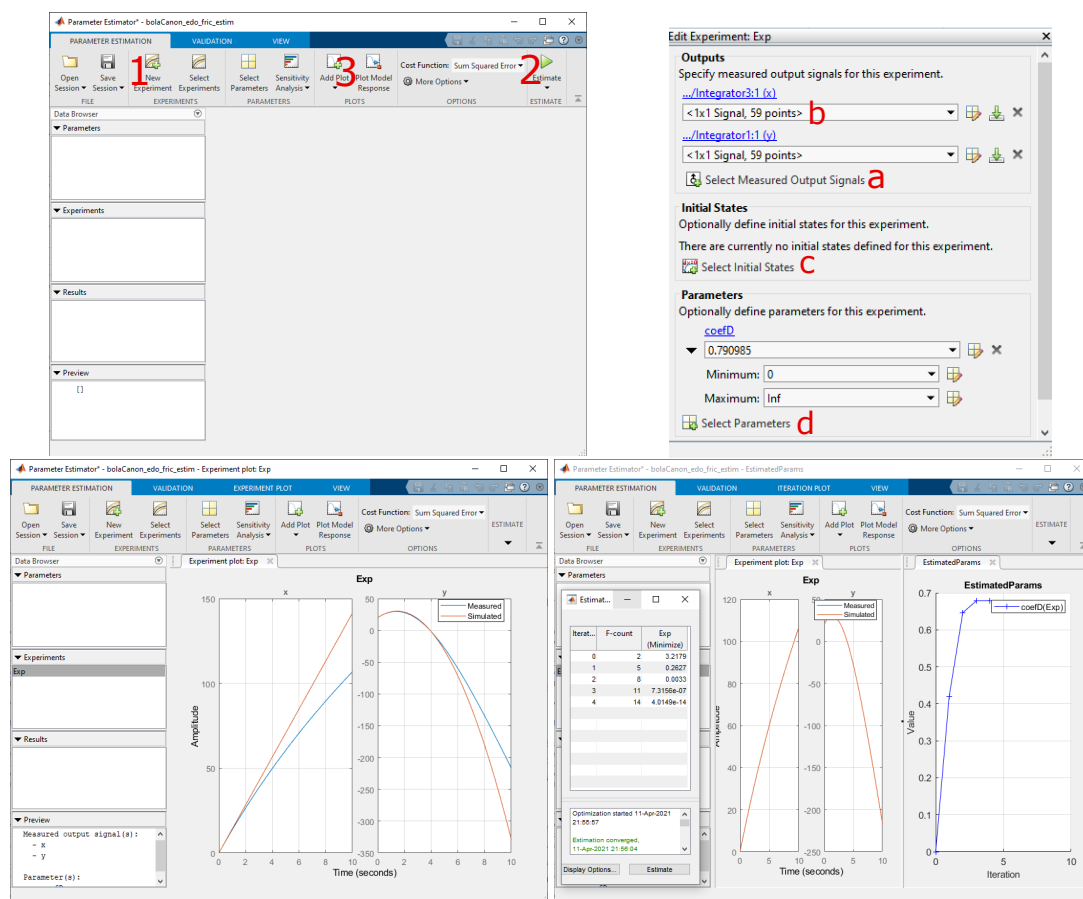


Figura 7: Modelo para la resolución del ejercicio 1 para determinar el coeficiente aerodinámico de una bala de cañón.

Ejercicio 2. Considera el mismo modelo que en el ejercicio anterior. Sin embargo, ahora desconocemos también la velocidad inicial de la bala v_0 y el ángulo de disparo del cañón θ . Como en el caso anterior la densidad del aire es de 0.074,

el área que presenta la bala es de 0.2. Hemos realizado mediciones de la altura y distancia recorrida por la bala a lo largo del tiempo y hemos obtenido los valores del fichero `xyCanon2.mat`.

Determina el coeficiente aerodinámico C_D de la bala así como su velocidad inicial v_0 y el ángulo del cañón θ .

Para realizar este ejercicio deberemos incorporar las condiciones iniciales a nuestro sistema de parámetros. Sin embargo, las condiciones iniciales de los integradores no son directas. Así pues, deberemos incorporarlas configurando el bloque Integrator. En le vídeo “Resolución del ejercicio 2” podrás ver cómo configurar el bloque de integración para aceptar condiciones iniciales así como la solución completa a este ejercicio.



Accede al vídeo: Resolución del ejercicio 2

5.5 Referencias bibliográficas

Borghi, R. (2014). On newton’s shell theorem. *European Journal of Physics*, 35.